

Resiliencia del pipeline ante corte de cuota Claude

Spike de validación · Issue [#3251](https://github.com/intrale/platform/issues/3251) · Ejecutado 2026-05-16 · Worktree platform.agent-3251-pipeline-dev .

Este documento responde la pregunta operacional: **¿Qué pasa con el pipeline V3 si la cuota Anthropic se agota un sábado a la madrugada?** El esquema multi-provider (issues #3198, #3221, #3233, #3235) declara fallbacks por skill, pero hasta este spike nunca lo habíamos probado en runtime. La conclusión corta es que la red de respaldo es una **ilusión de cobertura**: el dispatcher rutea correctamente al fallback, pero los handlers reales son stubs y el Telegram Commander es un punto único de falla.

> El alcance del spike es investigación. **No se entregan fixes acá.** Los hallazgos se materializan como issues hijos al cierre (CA-8).

Veredicto ejecutivo

De los 7 componentes del pipeline con relevancia LLM medidos, **3 sobreviven sin Claude** (Pulpo core, listener-telegram, skills determinísticos) y **4 mueren** (handlers stub openai-codex/groq/gemini-google/cerebras + Telegram Commander). Los issues críticos son la falta de runtime real en los handlers de fallback ([#3198](https://github.com/intrale/platform/issues/3198) / [#3076](https://github.com/intrale/platform/issues/3076)) y el hardcode de provider: 'anthropic' en ejecutarClaude del Commander.

Matriz de resultados (CA-1) — ordenada por severidad operacional

Componente	Comportamiento observado	Impacto operacional	Acción de mitigación	Issue de seguimiento
Telegram Commander (pulpo.js:6800 ejecutarClaude)	<code>spawn(CLAUDE_LAUNCHER.cmd, ...)</code> directo a Claude Code CLI. NO llama <code>resolveSpawnWithFallback</code> . <code>env-isolation</code> hardcodea <code>provider: 'anthropic'</code> . Detector de cuota hardcodea <code>cmdProvider = 'anthropic'</code> .	[CRÍTICO] El único canal humano↔pipeline queda mudo cuando Claude cae. Sin <code>/ghostbusters</code> , <code>/reset</code> ni pausa remota desde Telegram. Ventana ciega para el operador hasta que vuelva Claude.	(a) Path mínimo en <code>ejecutarClaude</code> que use <code>resolveSpawnWithFallback</code> para el skill virtual <code>commander</code> . (b) Modo degradado sin LLM: comandos pre-definidos (<code>/status</code> , <code>/ghostbusters</code> , <code>/reset</code> , <code>/pause</code>) parseados con regex y servidos vía filesystem queue, sin spawnear Claude.	[#3253] (https://github.com/intrale/platform/issues/3253) (abierto por este spike, <code>priority:critical</code>)
Handler openai-codex (providers/openai-codex.js)	Stub <code>_notImplemented</code> . Dispatcher SÍ lo selecciona como fallback (log  doc:#3251 <code>primary=anthropic</code> <code>gated</code> , usando <code>fallback="openai-codex" (índice 0)</code> , pero <code>handler.buildSpawn()</code> tira: <code>Provider "openai-codex" no</code>	[CRÍTICO] La red de respaldo del 80% de los skills (todos los que ponen <code>openai-codex</code> primero en <code>fallbacks[]</code>) está apagada. El archivo de	Entrega real del wrapper Codex CLI según diseño de [#3076] (https://github.com/intrale/platform/issues/3076). Mientras tanto, evitar mensajes engañosos: el dispatcher emite event <code>fallback_selected</code> antes del throw, dando la falsa sensación de cobertura.	[#3076] (https://github.com/intrale/platform/issues/3076) (existente) + [#3255] (https://github.com/intrale/platform/issues/3255) (recomendación de mensa

	<code>está implementado</code> <code>todavía (operación:</code> <code>buildSpawn).</code>	trabajo rebota a <code>pendiente/</code> con motivo claro, NO crashea el pulpo, pero NO se ejecuta el agente.	
--	---	--	--

Handler <code>groq</code> (<code>providers/groq.js</code>)	Stub <code>_notImplemented</code> . Mismo patrón: selección del dispatcher + throw en buildSpawn.	<code>[CRÍTICO]</code> Segundo en la cadena de fallback para la mayoría de skills (después de <code>openai-</code> <code>codex</code>). Si Claude + Codex caen, el siguiente intento también muere.	Entrega real del wrapper Groq (REST OpenAI- compat) por <code>[#3198]</code> (https://github.com/intrale/platform/issues/3198).	<code>[#3198]</code> (https://github.com/intrale/platform/issues/3198) (existente)
--	---	---	---	--

Handler <code>gemini-google</code> (<code>providers/gemini-google.js</code>)	Stub <code>_notImplemented</code> . Idem.	<code>[ALTO]</code> Tercero en cadena para <code>android-</code> <code>dev</code> , <code>web-</code> <code>dev</code> , <code>qa</code> , <code>po</code> , <code>ux</code> , <code>perf</code> .	Entrega real del wrapper Gemini por <code>[#3198]</code> (https://github.com/intrale/platform/issues/3198).	<code>[#3198]</code> (https://github.com/intrale/platform/issues/3198)
--	---	--	--	---

Handler <code>cerebras</code> (<code>providers/cerebras.js</code>)	Stub <code>_notImplemented</code> . Idem.	<code>[ALTO]</code> Último en cadena para casi	Entrega real del wrapper Cerebras por <code>[#3198]</code> (https://github.com/intrale/platform/issues/3198).	<code>[#3198]</code> (https://github.com/intrale/platform/issues/3198)
--	---	--	--	---

		todos los skills. Sería el último cartucho antes de quedarse sin red.	
--	--	---	--

Dispatcher <code>resolveSpawnWithFallback</code> (<code>dispatch-with-fallback.js:227</code>)	✅ Selecciona correctamente el primer fallback no-gateado del array <code>fallbacks[]</code> cuando el primario está marcado por <code>quota-exhausted.json</code> . Emite event audit <code>fallback_selected</code> , notifica via filesystem queue Telegram y devuelve <code>{provider, model, handler, source:'fallback', crossProvider:true}</code> . Logs reales obtenidos durante E1 y E2 más abajo.	[BAJO] La lógica de dispatch funciona como diseñada. Su efecto operacional es nulo porque los handlers downstream son stubs, pero la rama está lista para cuando #3198/#3076 entreguen runtime.	Ninguna acción de mitigación en el dispatcher en sí. Evaluar agregar warning preventivo al boot del pulpo si detecta que <code>fallbacks[]</code> apunta a handlers stub.	[#3254] (https://github.com/intrale/platform/issues/3254) (recomendación abierta por este spike)
--	---	--	--	---

Detector estructurado de cuota (<code>quota-exhausted.js:_detectAnthropic</code>)	✅ Matchea correctamente el shape real del stream-json de Anthropic (<code>evt.type === 'result' && evt.is_error === true && evt.error_type ∈ ['usage_limit_error', 'weekly_quota_exhausted', 'snapshot_threshold_90']</code>). El flag <code>.pipeline/quota-exhausted.json</code> se escribe CON <code>provider, model, resets_at, pattern_matched</code> .	[BAJO] Funciona. La causa real ("usage_limit_error") queda registrada y el dashboard puede mostrarla — no cae a "error genérico".	Ninguna. Pre-condición para que el resto del flow funcione.	—
--	---	--	--	---

Pulpo core loop (intake/outtake/dispatch, <code>pulpo.js</code> minus <code>ejecutarClaude</code>)	✅ Es Node + filesystem-como-estado. Las 5 referencias a Claude en <code>pulpo.js</code> viven en <code>detectClaudeLauncher</code> (detector pasivo), <code>ejecutarClaude</code> (Commander), <code>_brazoCommanderInner</code> (Commander) y <code>cmdProponer</code> (Commander/historias). El intake desde GitHub, el outtake hacia las carpetas de fase, la promoción y los rebotes son decisiones determinísticas.	[N/A] El Pulpo NO consume tokens. CA-3 cerrado: no es SPoF de Claude.	Ninguna. Punto fuerte del diseño.	—
---	---	--	--	---

<code>listener-telegram.js</code> (long-polling Telegram)	✓ Node + <code>https</code> + <code>filesystem</code> queue puros. Cero <code>require</code> de Claude/Anthropic/OpenAI.	[N/A] Inmune al corte de Claude. Sigue escribiendo <code>servicios/telegram/pendiente/*.json</code> mientras llegan mensajes; el Commander los procesa cuando vuelva Claude.	Ninguna. El backlog se acumula en <code>filesystem</code> y se drena al recuperarse.	—
Skills determinísticos (<code>build</code> , <code>tester</code> , <code>linter</code> , <code>delivery</code>)	✓ Node puro (<code>.pipeline/skills-deterministic/*.js</code>). El handler <code>deterministic</code> ignora flag de cuota en el dispatcher (línea 269 de <code>dispatch-with-fallback.js</code>).	[N/A] Siguen ejecutando sin Claude. El pipeline puede compilar, testear, mergear y empaquetar sin red LLM.	Ninguna. Diseño correcto, ya validado en producción.	—

> **Lectura clave:** la columna [CRÍTICO] arriba muestra el costo real de no probar la red de respaldo antes. La fila más severa, **Telegram Commander como SPoF**, es la que rompe la primitiva más básica del modo degradado — el operador queda sin canal de control durante el outage, que es exactamente cuando más lo necesita.

Evidencia empírica (logs textuales del spike)

El harness `.pipeline/tools/spike-3251-cuota-claude.js` corre los 5 escenarios sin spawnear agentes reales (CA-7: minimizar tokens). Usa el dispatcher real, el módulo de cuota real y los handlers reales (incluidos stubs) para observar comportamiento end-to-end.

E1 — Claude API key inválida (cuota tipo "credenciales") → fallback selección + buildSpawn

```
setFlag result: { exhausted: true, provider: 'anthropic',
  model: 'claude-opus-4-7', resets_at: '2026-05-16T17:06:00.369Z',
  detected_at: '2026-05-16T16:06:00.370Z', pattern_matched: 'credit_balance_too_low' }
[log/lanzamiento] [📧 doc:#3251 primary=anthropic gated, usando fallback="openai-codex" (índice 0)
resolveSpawnWithFallback("doc"):
  provider=openai-codex model=gpt-5-codex source=fallback
  gated=false fallbackUsed={ index: 0, provider: 'openai-codex' }
  chainTried=[anthropic → openai-codex] crossProvider=true

Probando handler.buildSpawn() del fallback resuelto:
  ● buildSpawn throw: [agent-launcher/openai-codex] Provider "openai-codex" no está implementado todavía (operación: buildSpawn).
```

Lectura: el dispatcher hace su trabajo (selección, audit, notificación) pero el handler downstream aborta. Ilusión de cobertura confirmada.

E2 — Rate limited 429 → mismo patrón, sin retry loop

```
[log/lanzamiento] [📧 guru:#3251 primary=anthropic gated, usando fallback="openai-codex" (índice 0)
  provider=openai-codex source=fallback gated=false
  chainTried=[anthropic → openai-codex]
  ● buildSpawn de openai-codex throw: [agent-launcher/openai-codex] Provider "openai-codex" no está implementado todavía (operación: buildSpawn).
```

Lectura: el flag se evalúa una sola vez por intento; no hay retry loop infinito. El watchdog del pulpo trata el throw como infra failure y mueve el archivo a `pendiente/` con motivo accionable. ✓

E3 — Cuota mensual exhausta (shape real Anthropic) → flag con causa real

```
detectQuotaError: matched=true errorType=usage_limit_error
quota-exhausted.json:
  pattern_matched=usage_limit_error provider=anthropic
  resets_at=2026-05-16T22:06:00.371Z
```

Lectura: el detector estructurado matchea el shape real del stream-json (`evt.type === 'result' && evt.is_error === true && evt.error_type === 'usage_limit_error'`). El dashboard tiene la causa real disponible — no es "error genérico". ✓

E4 — Pulpo orquestador sin LLM en su core loop

```
Referencias LLM en pulpo.js: 5
Llamadas LLM por función contenedora:
  detectClaudeLauncher: 1  (detector pasivo, no consume tokens)
  cmdProponer: 1          (proposición de historias del Commander)
  ejecutarClaude: 1       (Commander)
  _brazoCommanderInner: 2 (Commander)
```

Lectura: las 5 referencias caen en funciones del Commander/historias. El core loop (intake desde GitHub, dispatch a fases, ejecución del watchdog, promoción a `procesado/`, rebotes) es Node + filesystem. **CA-3 cerrado:** el Pulpo NO es SPoF de Claude. ✓

E5 — Telegram Commander hardcoded a Anthropic

```
commander → spawn directo de CLAUDE_LAUNCHER:      true
commander → env-isolation hardcodeda provider 'anthropic': true
commander → quota-detector hardcodeda cmdProvider 'anthropic': true
commander → usa resolveSpawnWithFallback:           false
veredicto:                                          SPoF confirmado
```

Lectura: confirma el hallazgo pre-spike de guru. `ejecutarClaude` (líneas 6800-6912 de `pulpo.js`) NO pasa por el dispatcher de fallback. Tres puntos hardcodedan Anthropic: el spawn (línea 6847-6856), la env-isolation (líneas 6832), y el detector de cuota del Commander (líneas 6925-6940). **CA-4 cerrado con veredicto (a): bug crítico confirmado.** ●

Pre-condiciones validadas (CA-2)

- **agent-models.json** carga **20 skills** en runtime (delta vs "17" mencionado en CA-2 del PO: refleja iteraciones posteriores al sign-off Leo 2026-05-15 / [#3236](https://github.com/intrale/platform/issues/3236); reportado como nota, no falla CA-2).
- 16 skills usan `anthropic` como primary con fallbacks declarados. 4 (`build` , `tester` , `linter` , `delivery`) SON `deterministic` .
- Tabla hardcoded de providers en `resolve-provider.js` : `anthropic` , `openai-codex` , `gemini-google` , `groq` , `cerebras` , `deterministic` — los 6 cargan, pero 4 son stubs.

```
backend-dev  primary=anthropic  fallbacks=[openai-codex,groq,cerebras]
pipeline-dev primary=anthropic  fallbacks=[openai-codex,groq,cerebras]
android-dev  primary=anthropic  fallbacks=[openai-codex,groq,gemini-google,cerebras]
web-dev      primary=anthropic  fallbacks=[openai-codex,groq,gemini-google,cerebras]
build        primary=deterministic fallbacks=[(none)]
tester       primary=deterministic fallbacks=[(none)]
security     primary=anthropic  fallbacks=[openai-codex,groq,cerebras]
qa           primary=anthropic  fallbacks=[openai-codex,gemini-google,groq,cerebras]
review       primary=anthropic  fallbacks=[openai-codex,groq,cerebras]
po           primary=anthropic  fallbacks=[openai-codex,gemini-google,groq,cerebras]
ux           primary=anthropic  fallbacks=[openai-codex,gemini-google,groq,cerebras]
doc          primary=anthropic  fallbacks=[openai-codex,groq,cerebras]
planner      primary=anthropic  fallbacks=[openai-codex,groq,cerebras]
guru         primary=anthropic  fallbacks=[openai-codex,groq,cerebras]
ops          primary=anthropic  fallbacks=[openai-codex,groq,cerebras]
perf         primary=anthropic  fallbacks=[openai-codex,gemini-google,groq,cerebras]
auth         primary=anthropic  fallbacks=[openai-codex,groq,cerebras]
refinar      primary=anthropic  fallbacks=[(none)]
linter       primary=deterministic fallbacks=[(none)]
delivery     primary=deterministic fallbacks=[(none)]
```

Plan de mitigación priorizado (CA-5)

Prioridad CRÍTICA — issue hijo abierto por este spike

- **[#3253](https://github.com/intrale/platform/issues/3253)** — **[pipeline] Telegram Commander es SPoF en Claude — habilitar multi-provider o modo degradado sin LLM.**

Mitigación mínima viable (orden de costo creciente):

- **(a) Modo degradado sin LLM:** comandos pre-definidos parseados con regex en `_brazoCommanderInner` (`/status` , `/ghostbusters` , `/reset` , `/pause` , `/quota`). Si llega un mensaje que no matchea regex, responder con texto fijo "Claude está caído, comandos disponibles: ...". Cero tokens, cero dependencia LLM, canal de control siempre vivo.
- **(b) Multi-provider real:** ejecutarClaude pasa por `resolveSpawnWithFallback({ skill: 'commander', ... })`. Requiere agregar `commander` como skill en `agent-models.json` con su propia `fallbacks[]`. Bloqueado por #3076/#3198 (handlers reales).
- Recomendación: empezar por (a) — desbloquea hoy. (b) se hace cuando #3198 entrega runtime.

Prioridad CRÍTICA — issues existentes a cerrar antes de confiar en la red

- **[#3076](https://github.com/intrale/platform/issues/3076)** — H3 multi-provider entrega del wrapper Codex CLI real. Sin esto, el primer fallback de la mayoría de skills es decorativo.
- **[#3198](https://github.com/intrale/platform/issues/3198)** — runtime real de groq / gemini-google / cerebras. Sin esto, el resto de la cadena de fallback es decorativa.

Prioridad ALTA — issues hijos no bloqueantes abiertos por este spike

- **[#3254](https://github.com/intrale/platform/issues/3254)** — **[recomendación][pipeline] warning preventivo al boot si fallbacks[] apunta a handlers stub.** El pulpo podría detectar al cargar `agent-models.json` que un fallback declarado apunta a un handler stub y emitir un warning visible en logs + Telegram. Hoy la red declara cobertura que no existe; un warning explícito al boot evita la ilusión. (Etiquetado `needs-human` + `priority:low` por protocolo del PO.)
- **[#3255](https://github.com/intrale/platform/issues/3255)** — **[recomendación][pipeline] dispatcher debe distinguir fallback stub al loggear fallback_selected.** Cuando `resolveSpawnWithFallback` selecciona un fallback que va a tirar `_notImplemented`, el log debería matizar con `(STUB - va a fallar)` o saltar al siguiente sin gastar el slot. Ahora mismo el operador ve "fallback ok" y el archivo aparece de nuevo en `pendiente/` minutos después sin contexto claro. (Etiquetado `needs-human` + `priority:low`.)

Prioridad MEDIA

- **[recomendación][pipeline] plantilla reutilizable de reportes de spike con audio narrado.** Detectado por UX en fase criterios. No bloqueante; lo dejamos registrado para próximos spikes si el formato de este resulta útil.

Trazabilidad de tokens del propio spike (CA-7)

El harness corre con Node puro: **cero tokens de pipeline consumidos**. No spawné agentes reales (no se ejecutaron `doc` , `guru` , ni ningún skill LLM). La validación E1-E5 se hace contra las funciones `setFlag` , `resolveSpawnWithFallback` , `detectQuotaError` directamente. El costo de tokens del propio agente que escribió este spike (`pipeline-dev` LLM) es la única partida — instrumentación detallada vive en `.pipeline/metricas/` si está habilitada.

Hallazgo no bloqueante: la captura per-escenario de tokens no está instrumentada en `.pipeline/metricas/` (solo agrega tokens por agente, no por escenario de prueba). Si en futuras corridas multi-provider queremos calibrar costo por proveedor + por escenario, hay que agregar un campo `scenario_id` al schema de métricas. Lo dejamos como mejora futura (no se abre issue ahora, va al backlog del próximo refactor de métricas).

Reproducibilidad

```
cd C:\Workspaces\Intrale\platform.agent-3251-pipeline-dev
node .pipeline/tools/spike-3251-cuota-claude.js
```

Output completo del run: en stdout. El harness limpia el flag `.pipeline/quota-exhausted.json` entre escenarios y al final. Aplica todos los guardrails de security definidos en la fase criterios:

- Keys sintéticas obvias (`spike-3251` en `agent` y `rawExcerpt`).
- Scope local (no se exporta `ANTHROPIC_API_KEY` globalmente; el Commander queda vivo durante todo el spike).
- Cleanup automático del flag entre escenarios.
- Audit log local desactivado (`auditLogEnabled: false`) para no contaminar `.pipeline/logs/` con un evento sintético.

Conclusión

El spike confirma de forma empírica el riesgo que la lectura previa de código ya sospechaba: tenemos infraestructura completa de fallback (config + validación + UI + dispatcher + audit + Telegram queue) pero el **kilómetro final está apagado**. Los handlers stub son honestos (`_notImplemented` con mensaje accionable), pero el efecto sistémico es engañoso: los logs dicen "fallback ok" mientras los archivos de trabajo rebotan a `pendiente/` y el operador, sin Commander, ni siquiera puede preguntarle al pipeline qué pasó.

Las dos acciones que más mueven la aguja son **(a) modo degradado del Commander sin LLM** (alto valor, bajo costo, desbloquea hoy) y **(b) cerrar #3076 + #3198** (alto valor, costo mayor, condición necesaria para que la red de respaldo sea real).